

Week 3 - Day 12b Supplementary: More Quick Sort

For **Quicksort**, pivot selection can be critical in determining the overall algorithmic performance - it can make the difference between the algorithm running in $O(n \log n)$ (logarithmic) or $O(n^2)$ (quadratic). For example, if the pivot selection algorithm always selects the last value of the list and the list is already sorted, then the resultant partitions will be imbalanced. Lets try a quick sort with a different pivot selection technique to avoid such an issue!

First, import the below packages.

```
import random
```

Exercise 1: Selecting Pivot []

One strategy for selecting a pivot is to select a random element from the list. Implement this strategy in the function `select_random_pivot_index` to select a random index from the list `lst`. We can later for a Quicksort implementation using a random pivot for each partition.

```
# TODO: Your code here
def select_random_pivot_index(lst):
    pass

# Use this code cell to test your solution
n = 5
test_data = list(range(5))
for _ in range(100):
    assert 0 <= select_random_pivot_index(test_data) < n

print("All tests passed. [ ]")
```

Exercise 2: Partitioning Lists []

Now that we have our random pivot selection function, lets write another function which partitions our list around this value. The function `partition` will partition `lst` into two lists, one where all the elements are smaller than the value at `pivot_index` and another with all values greater or equal. Do not include the pivot element itself in any of the sublists.

```
# TODO: Your code here
def partition(lst, pivot_index):
    left = []
    right = []

    # Partition the list here
```

```

    return left, right

# Use this code cell to test your solution
assert partition([3,57,0,4,23,9,5], 4) == ([3, 0, 4, 9, 5],[57])
assert partition([1,2,3,4,5], 2) == ([1,2],[4,5])
assert partition([10,5,8,2,7], 0) == ([5,8,2,7],[1])
assert partition([1,1,1,1], 1) == ([],[1,1,1])
assert partition([-3,0,-1,4,-2], 3) == ([-3,0,-1,-2],[1])

print("All tests passed. ☑")

```

Exercise 3: Quicksort [] []

Great, now that we are able to select a pivot and partition our data around this value lets implement the quicksort algorithm using these helper functions. Implement this recursively.

```

# TODO: Your code here
def quicksort(lst):
    pass

# Use this code cell to test your solution
assert quicksort([]) == []
assert quicksort([42]) == [42]
assert quicksort([1,2,3,4,5]) == [1,2,3,4,5]
assert quicksort([5,4,3,2,1]) == [1,2,3,4,5]
assert quicksort([3,57,0,4,23,9,5]) == [0, 3, 4, 5, 9, 23, 57]

print("All tests passed. ☑")

```

What is the worst case time complexity for the runtime of the quicksort algorithm?

Exercise 4: Quickselect [] [] []

The aforementioned pivoting and partitioning techniques are not unique to the Quicksort algorithm. In fact, there is a selection algorithm called **Quickselect** that can be used to select the k-th smallest element from an unordered list. The general algorithm can be described in the following steps ...

- Select a pivot (e.g. randomly selected)
- Partition elements around the pivot
- Determine whether the k-th smallest element has been found, if so return this value
- Otherwise, search one half (either left or right) depending on the k-th smallest search value relative to the current pivot

Try implementing the `quickselect` function recursively, and remember to reuse the helper functions `select_random_pivot_index` and `partition_lists`.

```

# TODO: Your code here
def quickselect(lst, k):
    pass

# Use this code cell to test your solution
assert quickselect([1], 1) == 1
assert quickselect([1, 2, 3, 4, 5], 1) == 1
assert quickselect([1, 2, 3, 4, 5], 5) == 5
assert quickselect([3, 2, 1, 5, 6, 4], 2) == 2
assert quickselect([5, 3, 7, 9, 6, 3, 2, 6, 8, 97, 4, 0], 8) == 6
assert quickselect([5, 3, 7, 9, 6, 3, 2, 6, 8, 97, 4, 0], 5) == 4
assert quickselect([5, 3, 7, 9, 6, 3, 2, 6, 8, 97, 4, 0], 10) == 8

print("All tests passed. 🎉")

```

What is the worst case time complexity for the runtime of the quickselect algorithm?

Challenge: What is the best case runtime for quickselect?

Congratulations 🎉